

EXHALE: Solver and Numerics Log

(numerical-method changes vs. the original ATES)

*This document is the single, complete record of the **solver / numerics** changes in EXHALE relative to the original ATES (ATES/ATES-Code-main): time integration of the stiff source terms, the convergence algorithm, the radial-grid convergence window, the advection post-processor's solve, and the nonlinear solvers (ionization equilibrium and the per-cell energy equation). It excludes the physics changes (metals, cooling, opacity, charge exchange, excited hydrogen, $\text{Ly}\alpha$), which are documented in Update_EXHALE and Update_EXHALE_early_phase. It absorbs in full the former energy_semi_implicit_solver.tex (§2.) and the solver portion of convergence_fix_and_validation.tex (§1.); those standalone memos are superseded by this file. The 2026-06-10/11 steady-state Newton–Krylov overhaul is summarized in §10. but documented in full in its own memo, steady_solver_memo.pdf. Sections are in chronological order.*

Contents

1. Convergence-algorithm fix (2026-06-02)	1
1.1 Overview	1
1.2 Symptom	1
1.3 Diagnosis	1
1.4 Fix	2
1.5 Verification	3
2. Semi-implicit energy solver (2026-06-03)	3
2.1 Introduction	3
2.2 Mathematical formulation	3
2.3 Original explicit solver (before the update)	4
2.4 Numerical algorithm	4
2.5 Implementation	5
2.6 Verification and performance	6
3. Escape-radius / empty-window grid guard (Phase 4)	6
4. Breathing-base advection guard (post-process, option c)	6
5. Brent for the scalar energy equation (Task 1, 2026-06-07)	7
5.1 Solver	7
5.2 Wiring	7
5.3 Validation	7

6. Analytic-Jacobian Newton for the ionization equilibrium (Task 2, 2026-06-07)	7
6.1 Newton solver module	8
6.2 Analytic Jacobians	8
6.3 Wiring and build	8
6.4 Validation	8
7. Input switches (new default, legacy hybrd1 optional)	9
8. Speed	9
9. Validation data (persisted) and reproduction	9
10. Steady-state Newton–Krylov solver and convergence overhaul (2026-06-10/11)	10

1. Convergence-algorithm fix (2026-06-02)

Absorbed in full from the former convergence_fix_and_validation.tex (its Phase 1/2 physics validation — opacity dispatcher, tabulated opacity, trace-metal smoke test — is intentionally not reproduced here; see Update_EXHALE_early_phase / Update_EXHALE).

1.1 Overview

This work fixed a non-terminating (effectively infinite) time-integration loop, discovered while running the first physical test case (HD 209458b; power-law SED, solar He/H, default numerics).

1.2 Symptom

A full run never terminated. The momentum-variation diagnostic `du` settled on a plateau and the run continued indefinitely:

iteration	du
26,905	0.62
52,239	0.077
667,569	0.0109 (still running)

1.3 Diagnosis

The main loop in `EXHALE_main.f90` terminated on a single criterion:

```
do while( .not.is_mom_const .or. force_start)
```

with `is_mom_const = (du < du_th)` and `du_th = 1e-3`.

The code also computes a steady-state criterion, `is_zero_dt = (dtu < dtu_th)` (the infinity-norm of the time-derivative of the conservative state), but **this flag was never used in the loop condition** — it appeared only in a commented-out diagnostic print.

For this setup the momentum non-uniformity floors at $du \approx 0.011$ (a transonic-wind numerical limit), which is $\sim 10\times$ above `du_th`. With `is_zero_dt` disconnected, no termination criterion could ever be satisfied. Measuring the time-derivative norm directly (starting from a near-converged IC) gave $dtu \approx 5.5 \times 10^{-8}$, i.e. the solution *had* reached steady state but the strict thresholds were physically unreachable.

1.4 Fix

Four changes, in `EXHALE_main.f90` and `parameters.f90`:

1. **Restore `is_zero_dt` in the loop condition.** The steady-state criterion is now an OR-branch of the termination test.
2. **Loosen `du_th`** from `1e-3` to `2e-2`. A momentum non-uniformity of 2% is accepted as converged, so the run stops as soon as the plateau is reached rather than waiting for an unreachable threshold. (This was a deliberate user choice; the value is a global parameter and affects all runs.)
3. **Add stall detection.** If the relative change in `du` stays below `stall_tol = 1e-6` for `N_stall = 2000` consecutive iterations, the run is declared stalled at a plateau (steady state) and stops. `stall_tol = 1e-6` was chosen so that a slowly-decreasing `du` (relative change $\sim 3 \times 10^{-6}$ in the descent phase) is *not* mistaken for a plateau — this prevents premature termination during convergence.
4. **Add a hard iteration cap** `count_max = 1e6` as a final backstop, and print the termination reason and final `du/dtu`.

The resulting loop condition is

```
do while( ( .not.is_mom_const .and. .not.is_zero_dt .and.  
           .not.is_stalled .and. count < count_max ) .or.  
           force_start )
```

Touched files. `src/modules/init/parameters.f90` (new flags/parameters, `du_th`); `EXHALE_main.f90` (loop condition, stall logic, termination report). The standard-output line now prints `count, du, dtu` (was `count, du`) for progress monitoring.

1.5 Verification

- **Plateau IC, loosened `du_th`:** starting from a near-converged profile, the run stops in 2 iterations via “momentum constant” ($du = 0.011 < 0.02$), with no stall wait.

- **Stall path (du_th=1e-3, before loosening):** a from-scratch run that previously never ended now stopped at iteration 554,485 via stall detection, with final $du = 0.011$ — exactly the floor, confirming `stall_tol` does not trigger prematurely during the descent.
- **Physics unchanged:** the post-processed profiles from the stall-terminated full run match the earlier do-only-PP run to 3 significant figures, confirming the convergence change does not alter the solution.

Caveat. $du_th = 2e-2$ is loose and global. Setups that genuinely converge to $du < 1e-3$ will now stop earlier, at 2% momentum non-uniformity. Tighten du_th (and rely on stall detection for the unreachable-threshold cases) if a given study needs a stricter steady state.

2. Semi-implicit energy solver (2026-06-03)

Absorbed in full from the former `energy_semi_implicit_solver.tex`. By replacing the explicit forward-Euler energy update with a constant-derivative 2-iteration Newton–Raphson scheme, the code resolves the numerical stiffness of radiative heating and cooling, achieving unconditional stability and a $2\times$ speedup on the main hydro loop.

2.1 Introduction

EXHALE solves the 1D spherically symmetric Euler equations coupled with a detailed chemical network for H, He, and metals. In highly irradiated or high-mass atmospheres the timescales of radiative heating ($\mathcal{H}$) and cooling ($\mathcal{C}$) are orders of magnitude shorter than the hydrodynamic cell-crossing time (the CFL limit):

$$\tau_{\text{cool}} = \frac{p}{(\gamma-1)\mathcal{C}} \ll \tau_{\text{CFL}} = \frac{\Delta r}{|v| + c_s}. \quad (1)$$

The original version of ATES integrated the energy source terms explicitly using a forward Euler step:

$$u_3^{n+1} = u_3^n + \Delta t (\mathcal{H} - \mathcal{C}(T^n)). \quad (2)$$

Because Δt is determined solely by the hydrodynamic CFL condition, this explicit treatment is highly unstable in the stiff regime. It introduces local high-frequency numerical oscillations in temperature and pressure, which delays global convergence (requiring $> 2 \times 10^5$ iterations) or triggers NaN crashes when pressure or temperature drops below physical limits. To overcome this, we implemented a robust, unconditionally stable semi-implicit energy solver that decouples the stiff thermal integration from the hydrodynamic time step.

2.2 Mathematical formulation

The 1D spherically symmetric energy conservation equation is

$$\frac{\partial E}{\partial t} + \frac{1}{r^2} \frac{\partial}{\partial r} [r^2 v (E + p)] = \mathcal{H} - \mathcal{C}, \quad (3)$$

with $E = \frac{1}{2} \rho v^2 + \frac{p}{\gamma-1}$. Using operator splitting, the hydrodynamic advection is performed first, followed by the local thermochemical source term; since ρ and v are constant during the source step the kinetic energy is fixed and the

equation reduces to one for T :

$$\frac{n_{\text{all}}}{\gamma-1} \frac{\partial T}{\partial t} = \mathcal{H} - \mathcal{C}(T), \quad (4)$$

where $n_{\text{all}} = n_{\text{tot}} + n_e$ is the total particle number density (adimensional). The backward-Euler discretization gives the implicit relation for T^{n+1} :

$$T^{n+1} - T^n = \Delta t \frac{\gamma-1}{n_{\text{all}}} (\mathcal{H} - \mathcal{C}(T^{n+1})). \quad (5)$$

2.3 Original explicit solver (before the update)

Formulation and time-step update. The advection step yields the intermediate total energy u_3^* ; the source integration then updates explicitly,

$$u_3^{n+1} = u_3^n + \Delta t (\mathcal{H}(T^n) - \mathcal{C}(T^n)), \quad (6)$$

after which $p^{n+1} = (\gamma-1)(u_3^{n+1} - \frac{1}{2}\rho(v^{n+1})^2)$ and $T^{n+1} = p^{n+1}/(n_{\text{all}}k_B)$.

Stability constraint and thermal stiffness. The time step is set purely by the hydrodynamic CFL condition,

$$\Delta t_{\text{CFL}} = C_{\text{CFL}} \min \left(\frac{\Delta r}{|v| + c_s} \right), \quad (7)$$

decoupled from the thermochemical timescales. For an explicit integration of $\frac{dT}{dt} = -\frac{T-T_{\text{eq}}}{\tau_{\text{cool}}}$ to remain stable requires $\Delta t \leq \tau_{\text{cool}} = n_{\text{all}}k_B T / [(\gamma-1)\mathcal{C}(T)]$. In highly ionized/irradiated zones τ_{cool} can be 10^{-3} – 10^{-6} of Δt_{CFL} , so the explicit limit is violated, giving:

- **Numerical oscillations:** high-frequency, unphysical T/p oscillations as the explicit solver overcorrects the source terms.
- **NaN crashes:** a large step with a high $\mathcal{C}(T^n)$ can over-cool a cell to negative pressure $\rightarrow$ negative- T exceptions $\rightarrow$ NaN.
- **Stalled convergence:** continuous minor oscillations at the wind base prevent the system from settling, slowing or stalling convergence.

2.4 Numerical algorithm

Define the residual

$$F(T^{n+1}) = T^{n+1} - T^n - \Delta t \frac{\gamma-1}{n_{\text{all}}} (\mathcal{H} - \mathcal{C}(T^{n+1})) = 0, \quad (8)$$

with derivative

$$\frac{\partial F}{\partial T} = 1 + \Delta t \frac{\gamma-1}{n_{\text{all}}} \frac{\partial \mathcal{C}}{\partial T}. \quad (9)$$

We solve $F = 0$ with an optimized Newton–Raphson incorporating two enhancements.

Non-negative derivative safeguard. In some regimes $\partial \mathcal{C} / \partial T < 0$; under a large Δt this could drive the Jacobian $\partial F / \partial T$ toward zero or negative, causing divergence or negative temperatures. We safeguard

$$\frac{\partial \mathcal{C}}{\partial T} \leftarrow \max \left(0, \frac{\partial \mathcal{C}}{\partial T} \right), \quad (10)$$

guaranteeing $\partial F / \partial T \geq 1$ always (unconditional stability, clamping any unphysical excursion).

Constant-derivative 2-iteration scheme. Evaluating $\mathcal{C}(T)$ is expensive (eval_cool: Gaunt factors, recombination, collisional ionization, metal-line cooling over all cells). Instead of re-evaluating the derivative each iteration, $\partial\mathcal{C}/\partial T$ is computed *once* at the start of the step from T^n and $\delta T = \max(10^{-5}, 10^{-5}T^n)$:

$$\frac{\partial\mathcal{C}}{\partial T} \approx \frac{\mathcal{C}(T^n + \delta T) - \mathcal{C}(T^n)}{\delta T}, \quad (11)$$

then 2 Newton iterations are taken with this fixed derivative:

1. Initialize $T^{[0]} = T^n$.
2. Evaluate $\mathcal{C}(T^{[0]})$ (call 1) and $\mathcal{C}(T^{[0]} + \delta T)$ (call 2) for the constant derivative.
3. **Iteration 1:** $T^{[1]} = T^{[0]} - F(T^{[0]})/(\partial F/\partial T)$.
4. Evaluate $\mathcal{C}(T^{[1]})$ (call 3).
5. **Iteration 2:** $T^{[2]} = T^{[1]} - F(T^{[1]})/(\partial F/\partial T)$.
6. Set $T^{n+1} = T^{[2]}$ and apply a physical floor $T \geq 0.01$ (adimensional).

This requires exactly 3 cooling evaluations per step. Compared to a standard 5-iteration Newton (10 evaluations) it cuts the cooling overhead by 70% and speeds up the whole run by $\sim 2\times$.

2.5 Implementation

The energy_semi_implicit module. Implemented in the new file

src/modules/time_step/energy_semi_implicit.f90

the subroutine solve_energy_semi_implicit(u, W, dt, heat, cool, f_sp) extracts local densities/temperatures, computes the adimensional n_e and n_{all} , runs the 2-iteration constant-derivative Newton, and updates u_3 and p .

Integration in EXHALE_main.f90. The explicit update

```
call W_to_U(W,u)
u(:,3) = u(:,3) + dt*(heat - cool)
```

was replaced with

```
call W_to_U(W,u)
call solve_energy_semi_implicit(u,W,dt,heat,cool,f_sp)
```

and the Makefile was updated to compile/link the new module with proper dependency ordering.

2.6 Verification and performance

End-to-end HD 209458b runs (run_compare_metals.sh) vs. the original explicit code:

- **Convergence speedup:** metals-ON converged in **71,993** steps (vs. **236,344** explicit), completing in 14 min 7 s ($\sim 33\%$ less wall-clock).
- **Physical accuracy:** $\log_{10} \dot{M} = 9.49061$ (vs. 9.49064); $T_{\max} = 6525.4$ K (vs. 6525.6); $T_{\min} = 1009.3$ K (exact) — the semi-implicit solver converges to the same physical state.
- **Robustness (metals OFF):** the explicit solver stalled at $du \approx 0.033$; the semi-implicit run reached the strict $du = 0.02$ threshold.

3. Escape-radius / empty-window grid guard (Phase 4)

Context: Update_EXHALE Phase 4. In `src/modules/init/define_grid.f90`, the convergence diagnostics (du , dtu) are computed over the radial window $[j_{\min} : N]$, where $j_{\min}$ is the first cell with $r \geq r_{\text{esc}}$ (the input Escape radius). When the escape radius lies *outside* the (L1-truncated Roche) domain, $r_{\text{esc}} > r_{\max}$ — e.g. WASP-121b Case D, $r_{\max} \sim 1.35 R_p < r_{\text{esc}} = 1.5 R_p$ — no cell satisfies $r \geq r_{\text{esc}}$, the DO loop leaves $j = N + N_g + 1$, and the window $[j_{\min} : N]$ is **empty**. `maxval/minval` over an empty slice return $\mp \text{HUGE}$, so $du = \infty$ while $dtu = -\text{HUGE} < dtu_{\text{th}}$, triggering a *spurious* “steady state” exit at step 0.

Guard. If $j_{\min} > N$, re-anchor $j_{\min}$ to the first cell with $r \geq 1 + \frac{1}{2}(r_{\max} - 1)$ (mid-domain) and print a loud WARNING instructing the user to set a smaller Escape radius `[R_p]:`. This converts a silent false-convergence into a visible, recoverable warning while still allowing the run to proceed on a sensible window.

4. Breathing-base advection guard (post-process, option c)

Full account and validation: Update_EXHALE, “Post-process advection guard at a breathing base”. Summary of the numerical change in `src/modules/post_process/post_process_adv.f90`:

For strongly Roche-filling planets the 1D wind is subsonic at L_1 and the dense lower atmosphere settles into a slowly recirculating (“breathing”) state with small *negative* (inflow) velocities and a stagnation point ($v = 0$) near the wind base. The advection post-processor re-solves the per-cell energy/ionization balance assuming an outflow (each cell upwinds from the next-inner cell); in the dense base the non-monotone metal line-cooling curve gives the energy equation a second, spurious *hot* root that the Newton/Powell solve can land on, and the upwind coupling cascades it outward (a sawtooth + overshoot spike in T).

Guard. Where the flow is *not* a clean outflow ($v \leq 0$) the advection correction is skipped and the converged equilibrium ionization and temperature are kept (gated on `pp_metal_on`; metals-off byte-identical). This breaks the upwind cascade at its source. The remaining genuine 1D limitation (the breathing base itself) is represented there by the smooth equilibrium solution. §5. makes the same per-cell energy solve robust by construction (Brent), removing the spurious-root failure mode entirely.

5. Brent for the scalar energy equation (Task 1, 2026-06-07)

`T_equation` (the per-cell post-process energy balance) is a single nonlinear equation in $x = T/T_0$. With metal cooling its residual is *non-monotone* and admits a second, spurious *hot* root that the general Newton/Powell solver (`hybrd1`) can land on — the root cause of the breathing-base spike (§4.). A bracketing solver that selects the lowest (physical) root removes the failure mode structurally. The scalar T solve runs only in the (single) post-process pass, so the scan cost is negligible.

5.1 Solver

Added to `T_equation.f90` (module `equation_T`):

- `Tres(xx, params)` — scalar residual wrapper around `T_equation` ($N = 1$).
- `solve_T_brent(params, x_guess, x_out, ok)` — a log-spaced upward scan from a low floor ($\max(0.05x_{\text{guess}}, 1\text{ K})$) to $4x_{\text{guess}}$, taking the **first sign change** (= lowest = physical root) and polishing with Brent. `ok=.false.` (no bracket) $\Rightarrow$ the caller falls back to the eq temperature.
- `brent_root(. . .)` — standard Brent's method on the bracket.

5.2 Wiring

In `post_process_adv.f90`'s per-cell T loop: when `pp_metal_on` (and `use_brent_solve`, §7.) call `solve_T_brent` (fall back to eq `T_in` if no bracket); otherwise the legacy `hybrd1` solve + the $2\times$ -band reject is used. Metals-off keeps the original `hybrd1` (monotone residual, byte-identical). The option-(c) $v \leq 0$ base $\rightarrow$ eq guard (§4.) is retained.

5.3 Validation

Rebuilt (gfortran). PP-only re-run on the converged Case B (feed the converged eq as the IC; Load IC + Do only PP): the wind cells ($v > 0$, 244 of them) match the `hybrd1` baseline to 6.5×10^{-12} (machine precision; Brent finds the same physical root), the base $[1.02, 1.10]R_p$ has 0 temperature sign-changes (smooth), and $\log_{10} \dot{M} = 13.38$ is unchanged. Brent reproduces the validated result to machine precision while *structurally* selecting the physical root (the $2\times$ -band reject is no longer needed).

6. Analytic-Jacobian Newton for the ionization equilibrium (Task 2, 2026-06-07)

All ionization systems were solved with MINPACK `hybrd1` (Powell hybrid with a *finite-difference* Jacobian). Task 2 supplies an *analytic* Jacobian with a self-contained damped-Newton solver, keeping `hybrd1` as a fallback (zero regression risk).

6.1 Newton solver module

New `src/modules/nonlinear_system_solver/newton_solver.f90`:

- `newton_dense(fcn, jac, n, x, params, ftol, info)` — damped Newton with a backtracking (Armijo) line search on $\|f\|_2$; the step $\mathbf{J} \delta \mathbf{x} = -\mathbf{f}$ is solved by Gaussian elimination with partial pivoting (`gauss_solve`). `info=1` on convergence (small Newton step or residual), 0 otherwise (singular Jacobian, no progress, NaN, or maxit).
- `solve_ieq(fcn, jac, n, x, params, tol, wa, lwa, used_newton)` — tries Newton; on *any* failure restores $\mathbf{x}$ and calls `hybrd1`. Run-wide counters `nt_calls/nt_fallback` are reported once by `EXHALE_main`.

6.2 Analytic Jacobians

`jac_system_H` (1×1) and `jac_system_HeH` (3×3) are direct. `jac_system_HeH_metals` is built structurally as

$$J(i, k) = [\text{photoion} + (\partial C_i / \partial x_{\text{local}}) n_e]_{\text{local}} + C_i \frac{\partial n_e}{\partial x_k},$$

a block-diagonal “direct” part plus a rank-1 n_e -coupling part. Charge exchange is added by `cx_add_to_jac` (new in `charge_exchange.f90`), the exact derivative mirror of `cx_add_to_fvec` (each rate $R = k_c DA$ is bilinear in two reactant densities, both linear in the unknowns). Absent elements and the unused upper stage of two-stage elements are pinned to identity rows. The He metastable-triplet system (`System_HeH_TR`) is left on `hybrd1` (no analytic Jacobian; it is mutually exclusive with metals and unused here).

6.3 Wiring and build

In `ionization_equilibrium.f90` the H-only, H/He, and H/He+metals `hybrd1` calls now go through the dispatcher `solve_ieq` (with the per-system `jac_*` callbacks); the triplet call is unchanged. `newton_solver.f90` was added to the Makefile. Builds clean (gfortran).

6.4 Validation

PP-only sweep over the converged Case B (one full `ioniz_eq` over all 504 cells):

- **Newton handled 504/504 solves (100%, zero fallback)** — the analytic Jacobian (H/He + 10 metals + charge exchange) converges on every cell.
- **A/B test at an identical state** (Newton vs. a forced-`hybrd1` run via `ATES_FORCE_HYBRD1=1`, so the one-step hydro drift cancels): `Ion_species` agrees to 7.2×10^{-7} over all stages, T to 3.5×10^{-4} — the Jacobian is correct.
- $\log_{10} \dot{M} = 13.38$ unchanged.
- A naive Newton-vs-baseline `Ion_species` comparison showed up to 63% on the trace neutral CI; that is the 0.26% one-step T drift amplified in a sensitive trace stage, *not* the solver (confirmed by the 7.2×10^{-7} A/B agreement).

7. Input switches (new default, legacy hybrd1 optional)

Both upgrades are the default and can be turned off per run via optional `input.inp` keywords (anywhere in the optional block):

keyword	default	effect when False
Newton solver: True/False	True	ionization equilibrium uses legacy MINPACK hybrd1
Brent solver: True/False	True	energy solve uses legacy hybrd1 + $2\times$ -band reject

Absent keywords keep the new solvers. The flags (`use_newton_ieq` / `use_brent_tsolve`, in `global_parameters`) are parsed in `input_read.f90`. The `ATES_FORCE_HYBRD1=1` environment variable also forces the hybrd1 ionization path for validation without editing the input.

8. Speed

Full-run wall-clock (hydro + radiation + ionization) over the converged Case B with Load IC + Force start (~ 2025 steps, identical trajectory), `OMP_NUM_THREADS=4`:

solver	steps	wall (run 1 / 2)	ms/step
Newton	2023	54.7 / 54.3 s	26.9
hybrd1	2027	67.8 / 68.9 s	33.7

The analytic-Jacobian Newton is **$\sim 25\%$ faster per step** ($\times 1.25$) than legacy hybrd1; the step counts match to 0.2% (fair per-step comparison). The `ioniz_eq` cell loop is *serial*, so the relative speedup grows with the thread count (hydro/radiation parallelize, the solve does not). This is on top of the $\sim 2\times$ hydro-loop speedup from the semi-implicit energy solver (§2.). Raw numbers in `WASP-121b/solver_validation/timing.txt`.

9. Validation data (persisted) and reproduction

The Task 1/2 comparison data lives in `WASP-121b/solver_validation/` (kept, not transient):

- `comparison.txt` — the results table (sections A/B/C).
- `newton/`, `hybrd1/` — the two PP-only runs (analytic-Jacobian Newton vs. the MINPACK hybrd1 reference) over the converged Case B, each with its full output/ and run.log.
- `baseline/` — a copy of the converged Case B (`WASP-121b/output/`).
- `run_validation.sh`, `compare.py` — regenerate everything with `cd WASP-121b/solver_validation && bash run_validation.sh`.
- `solver_comparison.ipynb` (built by `build_solver_nb.py`) — plots: (1) T , n , v and key ion densities for baseline/hybrd1/Newton overlaid (they coincide); (2) the pure solver difference Newton–hybrd1 vs. radius ($\sim$

10^{-6} – 10^{-7} over the physical region $[1.05, 2]R_p$); (3) the Brent _adv base temperature (smooth; machine-precision match in the wind).

- timing.txt — the speed table of §8..

Latest comparison.txt:

```
A. Newton vs hybrd1 (identical state -> pure SOLVER difference)
  eq T          max rel diff : 3.499e-04
  Ion_species   max rel diff : 7.186e-07 (worst: FeI)
B. Newton vs baseline (converged; includes one-step PP hydro drift)
  eq T          max rel diff : 2.578e-03
  Ion_species   max rel diff : 6.266e-01 (worst: CI, drift-amplified)
C. Brent _adv T vs baseline _adv (Task 1, scalar energy equation)
  wind (v>0)    max rel diff : 6.504e-12
  all cells     max rel diff : 3.499e-04
```

Section A is the meaningful solver test (identical state, so the one-step drift cancels); the 63% on the trace neutral CI in section B is that 0.26% T drift amplified, not the solver.

10. Steady-state Newton–Krylov solver and convergence overhaul (2026-06-10/11)

This is the largest solver change to date; it is summarized here for completeness of this log and **documented in full in steady_solver_memo.pdf** (development record, validation, and hardening), with user-facing usage in the user manual §2.4–2.5.

Why. The marching du stop (relative spread of $\rho v r^2$) is provably blind to the operator-split energy imbalance: states that satisfy $du < 10^{-3}$ can carry steady residuals $\|R\|_E \sim 30$ (“premature dip”), shifting $\dot{M}$ by $\sim 20\%$ (WASP-121b: $\log_{10} \dot{M}$ 13.631 premature vs. 13.711 converged). The fix is to measure convergence by the *steady residual* R of the discrete equations and, near the fixed point, to solve $F(Y) = 0$ directly.

What was added (all opt-in via input.inp keywords; every legacy default is byte-identical, regression-gated):

- *Automatic two-stage marching* (du_th [PLM, WENO3]): PLM warm-up, switch to WENO3 at the first threshold, stop at the second.
- *Residual monitor and residual-based stop* (Resid tol, Level tol): periodic max $\|R\|_\infty$ over mass / momentum / energy; criterion-independent of du .
- *Preconditioned JFNK steady solver* (src/modules/time_step/steady_newton.f90): matrix-free Newton–Krylov with right-preconditioned GMRES, colored-FD banded preconditioner (LAPACK dgbtrf/dgbtrs, $kl=ku=8$), PTC ($I/\Delta\tau + J$) with SER ramp, and the five ingredients each found via a localized stall: smooth

$\|D^{-1}F\|_2$ merit, diagonal scaling, smooth base valve (Valve eps), frozen WENO weights, and a non-monotone (Grippo) line search.

- *Production wiring* (Solver: Newton): marching warm-up with *du*-stops suspended until the monitored $\|R\|$ falls below the switch (5×10^{-2}), then the JFNK finish to *Resid tol* (10^{-3}), then standard outputs and post-processing. Validated end-to-end from a cold IC (WASP-121b: identical $\dot{M}$ to the warm-started reference; HD189733b: 33 s where marching spent >3 h).
- *Failure containment* (2026-06-11): best-iterate tracking, fail-fast after 15 outer iterations without a new best residual (*info*=2), and on failure EXHALE_main keeps the best iterate, disables the Newton mode, and resumes plain marching — an unconverged Newton state is never accepted as the final answer.

Ready-to-run configurations for every solver combination (legacy marching, two-stage, Newton, Newton-from-state, warm-seed IC, ...) are collected under *examples/* (user manual, Table 4).